

Agentic RAG with Web Search

1) Project Overview

The objective of this project is to develop an Agentic Retrieval-Augmented Generation (Agentic RAG) system capable of answering user queries using both internal knowledge and real-time web information. Traditional RAG systems rely only on a predefined knowledge base and cannot access newly published information available on the internet. As a result, they may provide incomplete or outdated responses.

To overcome this limitation, the proposed system combines Retrieval-Augmented Generation with autonomous AI agents and web search capabilities. The agents analyze the user query, determine the required information sources, retrieve relevant data from a vector database and the web, and generate a final response based on the collected context.

2) Proposed Solution

The proposed solution is an Agentic RAG system integrated with web search capabilities. The system combines the strengths of Large Language Models (LLMs), vector-based knowledge retrieval, and real-time internet search to provide accurate and up-to-date responses to user queries.

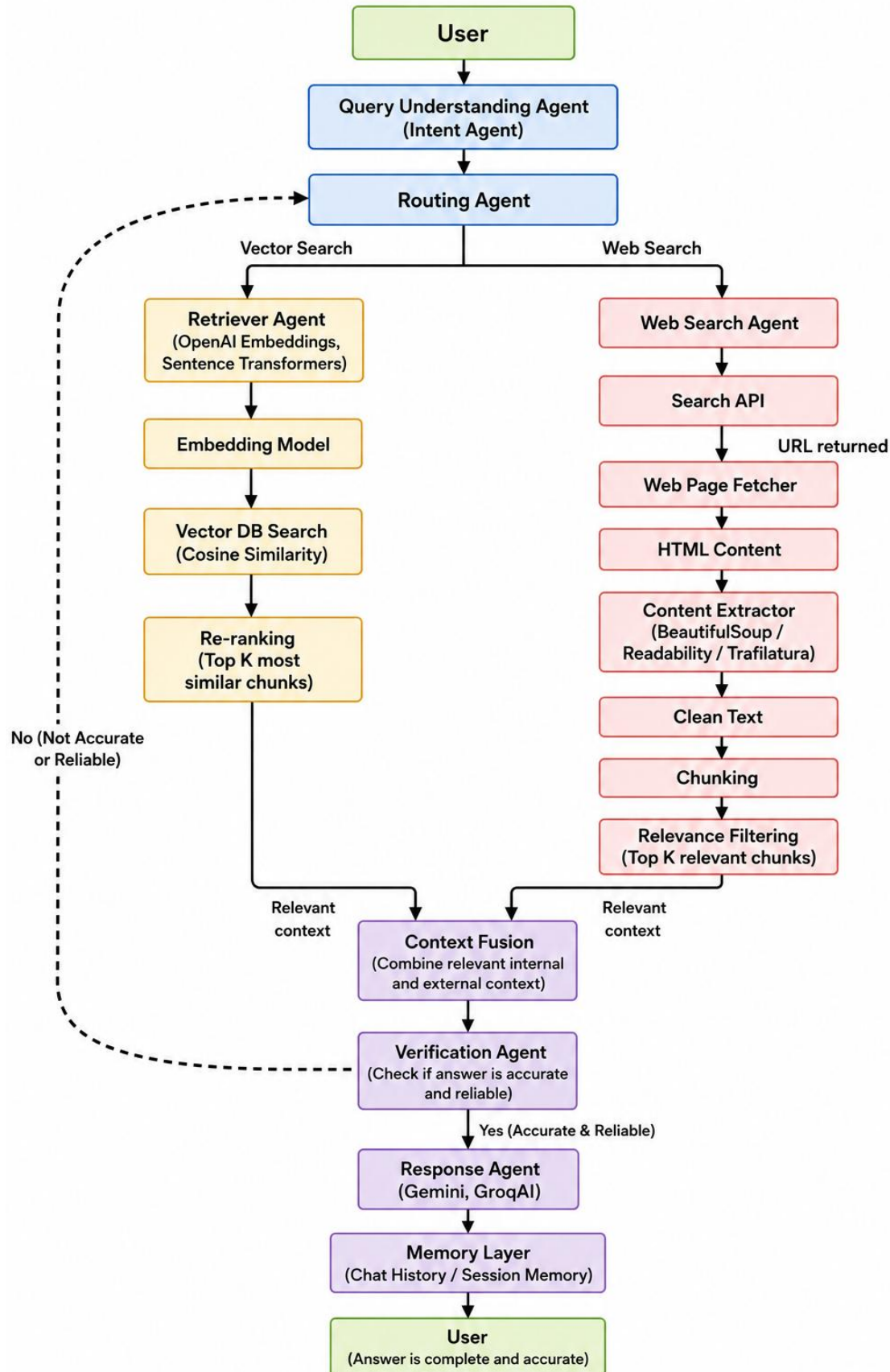
Instead of relying solely on a static knowledge base, the system uses multiple AI agents that collaborate to understand the user query, retrieve relevant information, search the web when required, analyze the collected data, and generate a final response. This enables the system to answer both domain-specific and time-sensitive questions more effectively.

The solution follows a hybrid retrieval approach:

- **Internal Retrieval:** Relevant information is retrieved from a vector database containing pre-processed and embedded documents.
- **Web Retrieval:** For queries requiring recent or external information, the system performs a web search using a search API and extracts relevant content from trusted web sources.
- **Context Fusion:** Information gathered from internal and external sources is combined into a unified context.
- **Response Generation:** The final context is provided to the LLM to generate a comprehensive and reliable response.

3) Workflow

The system follows a multi-stage workflow in which specialized agents collaborate to retrieve information, validate it, and generate a reliable response.



Step 1: User Query Submission

The workflow begins when a user submits a query to the system.

Step 2: Query Understanding

The Query Understanding Agent analyzes the query to identify:

- User intent
- Query type
- Information requirements
- Need for internal retrieval, web search, or both

Step 3: Routing Agent

The Routing Agent creates an execution strategy based on the query analysis and determines which information sources should be used.

Step 4: Internal Knowledge Retrieval

If internal knowledge is required:

- The Retriever Agent receives the query.
- The Embedding Model converts the query into a vector representation.
- The Vector Database performs similarity search using cosine similarity.
- Retrieved document chunks are re-ranked.
- The most relevant chunks are selected as internal context.

Step 5: Web Information Retrieval

If external or recent information is required:

- The Web Search Agent sends the query to a Search API.
- Relevant URLs are returned.
- The Web Page Fetcher retrieves webpage content.
- The Content Extractor extracts clean textual information from the HTML.
- The extracted content is chunked into smaller sections.
- Relevance filtering selects the most useful content as external context.

Step 6: Context Fusion

The Context Fusion Module combines:

- Internal context from the vector database
- External context from web search

Step 7: Verification

The Verification Agent evaluates whether the collected information is:

- Relevant
- Complete
- Accurate
- Reliable

If the information is insufficient or inconsistent, the workflow returns to the Routing Agent for additional retrieval and processing.

Step 8: Response Generation

Once the information is verified, the Response Agent uses a Large Language Model (LLM) such as Gemini, or Llama to generate the final response.

Step 9: Memory Storage

The generated response and conversation context are stored in the Memory Layer to support future interactions and maintain session continuity.

Step 10: Response Delivery

The final validated response is returned to the user.

4) Agent Design

The system follows a multi-agent architecture where each agent is responsible for a specific task within the workflow. This modular design improves scalability, maintainability, and decision-making capabilities.

Agent	Responsibility
Query Understanding Agent	Analyzes the user query, identifies intent, determines the information requirement, and extracts key objectives needed to answer the query.
Routing Agent	Creates the execution strategy for the query. It decides whether to perform vector retrieval, web search, or both, and coordinates the overall workflow.
Retriever Agent	Retrieves relevant information from the vector database. It converts the query into embeddings, performs similarity search, and returns the most relevant document chunks.

Web Search Agent	Retrieves external information from the internet. It interacts with the Search API, fetches webpages, extracts useful content, and prepares relevant web context.
Verification Agent	Validates the collected information to ensure the generated answer is accurate, reliable, and complete. If verification fails, the workflow is redirected back to the Planning Agent for additional retrieval.
Response Agent	Generates the final response using a Large Language Model (LLM) such as Gemini, or Llama based on the verified context.

5) Search API Options Considered

Search API	Advantages	Limitations
Tavily <i>(Recommended)</i>	Specifically designed for AI Agents and RAG systems, returns relevant search results with extracted content, easy integration with CrewAI and agent frameworks, developer-friendly API	Free tier has usage limits
Serper	Provides Google-quality search results, simple REST API, widely used in AI applications	Usually returns search results only; additional webpage fetching and content extraction may be required
Brave Search API	Good search quality, privacy-focused, AI-friendly API, straightforward integration	Smaller adoption in Agentic RAG projects compared to Tavily and Serper

6) LLM Options Considered

LLM	Advantages	Limitations
Gemini	Strong reasoning capabilities, large context window, good performance for retrieval and summarization tasks, generous free tier for development	Rate limits on free usage and occasional model restrictions
GroqAI	Extremely fast inference speed, low latency, supports open-source models (Llama, Gemma, etc.), developer-friendly APIs	Model quality depends on the underlying model being used; fewer advanced proprietary models compared to Gemini

7) Tech Stack

The following technologies can be used for developing the proposed Agentic RAG system. The final technology selection may vary depending on project requirements, scalability needs, and deployment constraints.

Component	Possible Technologies	Purpose
Frontend	Next.js, React.js	User interface for query submission and response visualization
Programming Language	TypeScript, JavaScript, Python	Application and agent development
Backend API	Next.js API Routes, FastAPI, Express.js	Handles requests and orchestrates the workflow
Agent Framework	CrewAI, LangGraph	Agent orchestration and workflow management
LLM (Response Agent)	Gemini, GroqAI	Response generation and reasoning
Embedding Model	OpenAI Embeddings, Sentence Transformers	Converts text into vector representations
Vector Database	Qdrant, Pinecone, ChromaDB	Stores embeddings and performs similarity search
Search API	Tavily, Serper, Brave Search API	Retrieves relevant web information
Content Extraction	BeautifulSoup, Readability, Trafilatura	Extracts clean text from web pages
Database	PostgreSQL	Stores application data and metadata
ORM	Prisma	Database access and schema management
Authentication	Auth.js (NextAuth)	User authentication and session management
Memory Layer	PostgreSQL, Redis	Session memory and conversation history
Deployment	Vercel, Docker, AWS	Application deployment and hosting

8) Project Structure

The project follows a modular architecture where agents, tools, APIs, retrieval components, and database-related code are separated into dedicated folders. This improves maintainability, scalability, and ease of development.

Agentic RAG - Recommended Project Structure

